

Структура научной презентации

Простейший вариант

Mehmet Efe Kantoz

30.08.2026

Содержание I

- 1 Цель работы
- 2 Задание
- 3 Теоретическое введение
- 4 Выполнение лабораторной работы
- 5 Выводы

Раздел 1

Цель работы

- Построить сеть Петри для пяти философов, моделируя захват и освобождение вилок.

- Построить сеть Петри для пяти философов, моделируя захват и освобождение вилок.
- Обнаружить состояние взаимной блокировки (deadlock), когда каждый философ взял одну вилку и ждёт вторую.

- Построить сеть Петри для пяти философов, моделируя захват и освобождение вилок.
- Обнаружить состояние взаимной блокировки (deadlock), когда каждый философ взял одну вилку и ждёт вторую.
- Провести имитационное моделирование (стохастическое и детерминированное) и выявить наличие deadlock.

- Построить сеть Петри для пяти философов, моделируя захват и освобождение вилок.
- Обнаружить состояние взаимной блокировки (deadlock), когда каждый философ взял одну вилку и ждёт вторую.
- Провести имитационное моделирование (стохастическое и детерминированное) и выявить наличие deadlock.
- Модифицировать сеть, чтобы предотвратить deadlock.

- Построить сеть Петри для пяти философов, моделируя захват и освобождение вилок.
- Обнаружить состояние взаимной блокировки (deadlock), когда каждый философ взял одну вилку и ждёт вторую.
- Провести имитационное моделирование (стохастическое и детерминированное) и выявить наличие deadlock.
- Модифицировать сеть, чтобы предотвратить deadlock.
- Проанализировать результаты и оформить отчёт с графиками и анимацией.

Раздел 2

Задание

Задание

- Создать рабочий каталог для кода.

Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.

Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.

Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.

Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:

Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
 - чистый код;

Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
 - чистый код;
 - jupyter notebook;

Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.

Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook.

Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.

Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.

Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:

Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:
 - чистый код;

Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:
 - чистый код;
 - jupyter notebook;

Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.

Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook с параметрами.

Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook с параметрами.

Раздел 3

Теоретическое введение

Первую идею сетей Петри Карл Адам Петри сформулировал в возрасте 13 лет для описания химических реакций. Официальной датой рождения теории считается 1962 год, когда он защитил докторскую диссертацию «Kommunikation mit Automaten» («Взаимодействие с автоматами») в Боннском университете. За свою работу он получил премию Тьюринга (ACM Turing Award) — высшую награду в области информатики

Сеть Петри есть математический аппарат для моделирования дискретных систем. Графически она представляется как двудольный ориентированный граф двух типов вершин: позиции (круги) и переходы (прямоугольники). - Позиции (Places) суть пассивные элементы, описывающие состояние системы (например, наличие ресурса или выполнение условия). - Внутри позиции могут находиться фишки (tokens) — неотрицательное целое число, указывающее на количество ресурсов. - Переходы (Transitions) суть активные элементы, описывающие события или действия системы. - Они могут срабатывать, изменяя состояние модели. - Дуги (Arcs) суть направленные соединения между позициями и переходами (но не между двумя позициями или двумя переходами), которые показывают, как состояние влияет на события и наоборот. - Маркировка (Marking) есть распределение фишек по позициям в определённый момент времени, то есть текущее состояние модели. - Смена маркировок происходит при срабатывании переходов в соответствии с определёнными правилами.

Теория сетей Петри, появившаяся как инструмент для анализа химических процессов, сегодня является мощным и наглядным математическим аппаратом. Она незаменима везде, где нужно описать параллельные, асинхронные и распределённые системы. В её основе лежат всего четыре элемента, а богатство поведения возникает из их комбинации.

Сети Петри используются не просто для моделирования, но и для проверки корректности системы. Для этого исследуются её важнейшие свойства: - Ограниченность (Boundedness). Количество фишек в любой позиции сети никогда не превысит некоторого заданного предела K . Если $K = 1$, сеть называется безопасной. - Активность (Liveness). Для любого перехода всегда существует достижимая маркировка, в которой он может сработать. Это гарантирует, что в системе не будет вечных блокировок. - Достижимость (Reachability). Можно ли из начальной маркировки M_0 попасть в некоторую желаемую маркировку M_k . - Сохраняемость (Conservation). Взвешенная сумма фишек по всем позициям остаётся постоянной (аналог закона сохранения массы или энергии).

Для изучения этих свойств разработаны специальные методы: - Граф (дерево) достижимости. Это фундаментальный метод. Строится ориентированный граф, вершинами которого являются все возможные маркировки, достижимые из начальной. Он наглядно показывает все возможные сценарии поведения системы. - Матричные уравнения. Поведение сети можно описать с помощью матриц инцидентности. Срабатывание перехода соответствует прибавлению вектора, что позволяет решать задачи достижимости алгебраическими методами. - Методы редукции. Сложная сеть упрощается путём замены стандартных подсетей (например, последовательных или параллельных фрагментов) на эквивалентные.

Задача «Обедающие философы» была сформулирована Эдсгером Дейкстрой в 1965 году как иллюстрация проблемы синхронизации в параллельных вычислительных системах. Она наглядно демонстрирует явления взаимной блокировки (deadlock) и голодания (starvation) при конкурентном доступе к разделяемым ресурсам. Это одна из самых известных задач, демонстрирующая проблемы параллелизма, в частности, взаимные блокировки (deadlocks) и состояние гонки (race conditions).

За круглым столом сидят N философов (обычно 5). Перед каждым философом стоит тарелка с едой. Между каждыми двумя соседними философами лежит одна вилка (или палочка для еды). Таким образом, количество вилок равно количеству философов (рис. 5.1). Правила поведения философов: - Философ может находиться в одном из трёх состояний: думает, голоден (хочет есть), ест. - Чтобы поесть, философу необходимы две вилки — та, что слева от него, и та, что справа. - Если философ не может получить обе вилки одновременно, он ждёт, пока они освободятся. - Поев, философ кладёт обе вилки обратно на стол и возвращается к размышлениям.

Ограничения:

- Вилками нельзя пользоваться одновременно двум философам.

Ограничения:

- Вилками нельзя пользоваться одновременно двум философам.
- Философ не может отнять вилку у соседа — только дожидаться, пока тот её положит.

Проблема взаимной блокировки (deadlock)

При наивной реализации (каждый философ сначала берёт левую вилку, затем правую) возникает тупиковая ситуация: если все философы одновременно возьмут левую вилку, то каждый будет ждать правую, которая уже занята соседом. В результате никто не может поесть — система замирает. Это и есть взаимная блокировка.

Для предотвращения deadlock предложены различные подходы: - Арбитр (официант) — вводится дополнительный процесс, который разрешает есть не более чем $N-1$ философам одновременно. - Асимметричное правило — философы с нечётным номером сначала берут левую вилку, затем правую, а чётные — наоборот. - Атомарный захват — философ берёт обе вилки одновременно (если они свободны) иначе не берёт ни одной. - Использование мьютексов и условных переменных в программных реализациях.

Задача «Обедающие философы» является классическим примером для моделирования с помощью сетей Петри. Каждый философ и каждая вилка представляются позициями, а переходы соответствуют действиям: «взять левую вилку», «взять правую вилку», «начать есть», «положить вилки». Сеть Петри наглядно показывает возникновение deadlock и позволяет экспериментировать с различными механизмами синхронизации, например, введением арбитра.

- Стохастичность: алгоритм Джиллеспи вносит случайность. При каждом запуске могут получаться разные траектории и разное время наступления deadlock. Для надёжности рекомендуется проводить несколько прогонов (например, с разными seed) и усреднять результаты.

- Стохастичность: алгоритм Джиллеспи вносит случайность. При каждом запуске могут получаться разные траектории и разное время наступления deadlock. Для надёжности рекомендуется проводить несколько прогонов (например, с разными seed) и усреднять результаты.
- Параметры N и $t_{\max}$: для $N=5$ deadlock в классической модели обычно наступает в течение 50 единиц времени.

- Стохастичность: алгоритм Джиллеспи вносит случайность. При каждом запуске могут получаться разные траектории и разное время наступления deadlock. Для надёжности рекомендуется проводить несколько прогонов (например, с разными seed) и усреднять результаты.
- Параметры N и $t_{\max}$: для $N=5$ deadlock в классической модели обычно наступает в течение 50 единиц времени.
- Если deadlock не обнаружился, можно увеличить $t_{\max}$.

- Стохастичность: алгоритм Джиллеспи вносит случайность. При каждом запуске могут получаться разные траектории и разное время наступления deadlock. Для надёжности рекомендуется проводить несколько прогонов (например, с разными seed) и усреднять результаты.
- Параметры N и $t_{\max}$: для $N=5$ deadlock в классической модели обычно наступает в течение 50 единиц времени.
- Если deadlock не обнаружился, можно увеличить $t_{\max}$.
- Сеть с арбитром.

- Стохастичность: алгоритм Джиллеспи вносит случайность. При каждом запуске могут получаться разные траектории и разное время наступления deadlock. Для надёжности рекомендуется проводить несколько прогонов (например, с разными seed) и усреднять результаты.
- Параметры N и $t_{\max}$: для $N=5$ deadlock в классической модели обычно наступает в течение 50 единиц времени.
- Если deadlock не обнаружился, можно увеличить $t_{\max}$.
- Сеть с арбитром.
- Теоретически она никогда не заходит в deadlock.

- Стохастичность: алгоритм Джиллеспи вносит случайность. При каждом запуске могут получаться разные траектории и разное время наступления deadlock. Для надёжности рекомендуется проводить несколько прогонов (например, с разными seed) и усреднять результаты.
- Параметры N и $t_{\max}$: для $N=5$ deadlock в классической модели обычно наступает в течение 50 единиц времени.
- Если deadlock не обнаружился, можно увеличить $t_{\max}$.
- Сеть с арбитром.
- Теоретически она никогда не заходит в deadlock.
- Все три скрипта вместе дают полное исследование: от численного моделирования и визуализации динамики до сравнительного анализа и наглядной анимации.

Раздел 4

Выполнение лабораторной работы

Выполнение лабораторной работы

Создаем файл в project/src с кодом модели, который будем использоваться в дальнейших скриптах(рис. 1).

```
DiningPhilosophers.jl X dining_philosophers.jl X
C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling> labs > lab05 > project > scripts > dining_philosophers.jl

1 using DrWatson
2 @quickactivate "project"
3 include(scrdir("DiningPhilosophers.jl"))
4 using .DiningPhilosophers
5 using DataFrames, CSV, Plots
6 N = 5
7 tmax = 50.0
8 println("=== Классическая сеть (без арбитра) ===")
9 net_classic, u0_classic, _ = build_classical_network(N)
10 df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
11 CSV.write(datadir("dining_classic.csv"), df_classic)
12 dead = detect_deadlock(df_classic, net_classic)
13 println("Deadlock обнаружен: $dead")
14 plot_classic = plot_marking_evolution(df_classic, N)
15 savefig(plotsdir("classic_simulation.png"))
16 println("\n=== Сеть с арбитражем ===")
17 net_arb, u0_arb, _ = build_arbiter_network(N)
18 df_arb = simulate_stochastic(net_arb, u0_arb, tmax)
19 CSV.write(datadir("dining_arbiter.csv"), df_arb)
20 dead_arb = detect_deadlock(df_arb, net_arb)
21 println("Deadlock обнаружен: $dead_arb")
22 plot_arb = plot_marking_evolution(df_arb, N)
23 savefig(plotsdir("arbiter_simulation.png"))
```

Базовый эксперимент

Создаем файл с базовым экспериментом, который выполняет основное моделирование и сравнение двух вариантов сети Петри: - классическая модель (без арбитра), в которой возможна взаимная блокировка (deadlock); - модифицированная модель с арбитром, которая должна предотвращать deadlock.(рис. 2).

```
DiningPhilosophers.jl X dining_philosophers.jl X
C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling> labs> lab05> project> scripts> dining_philosophers.jl

1 using DrWatson
2 @quickactivate "project"
3 include(scrdir("DiningPhilosophers.jl"))
4 using .DiningPhilosophers
5 using DataFrames, CSV, Plots
6 N = 5
7 tmax = 50.0
8 println("=== Классическая сеть (без арбитра) ===")
9 net_classic, u0_classic, _ = build_classical_network(N)
10 df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
11 CSV.write(datadir("dining_classic.csv"), df_classic)
12 dead = detect_deadlock(df_classic, net_classic)
13 println("Deadlock обнаружен: $dead")
14 plot_classic = plot_marking_evolution(df_classic, N)
15 savefig(plotsdir("classic_simulation.png"))
16 println("\n=== Сеть с арбитром ===")
17 net_arb, u0_arb, _ = build_arbiter_network(N)
18 df_arb = simulate_stochastic(net_arb, u0_arb, tmax)
19 CSV.write(datadir("dining_arbiter.csv"), df_arb)
20 dead_arb = detect_deadlock(df_arb, net_arb)
```


Запускаем данный скрипт (рис. 3).

```
julia> exit()
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts> julia dining_philosophers.jl
Activating project at 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project'
=== Классическая сеть (без арбитра) ===
Deadlock обнаружен: true
|
=== Сеть с арбитром ===
Deadlock обнаружен: false
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts> |
```

Рисунок 3: Запуск

Базовый эксперимент

Далее создаем все производные форматы(рис. 4).

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts> julia tangle.jl dining_philosophers.jl
Activating project at 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project'
Генерация из: dining_philosophers.jl
[ Info: generating plain script file from 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts\dining_philosophers.jl'
[ Info: writing result to 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts\dining_philosophers\dining_philosophers.jl'
      ✓ Чистый скрипт: C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts\dining_philosophers\dining_philosophers.jl
[ Info: generating markdown page from 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts\dining_philosophers.jl'
[ Info: writing result to 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\markdown\dining_philosophers\dining_philosophers.qmd'
      ✓ Quarto: C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\markdown\dining_philosophers\dining_philosophers.qmd
[ Info: generating notebook from 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts\dining_philosophers.jl'
[ Info: writing result to 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\notebooks\dining_philosophers\dining_philosophers.ipynb'
      ✓ Notebook: C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\notebooks\dining_philosophers\dining_philosophers.ipynb

Готово! Все файлы созданы.
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts>
```

Рисунок 4: Код модели

Запускаем файл notebook(рис. 5).

Анимация процесса

Создаем следующий скрипт, который сделает gif файл с анимацией процесса и наглядно продемонстрирует динамику работы сети петри(рис. 8).

```
=== Сеть с арбитром ===
Deadlock обнаружен: false
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts> julia dining_philosophers_animation
.jl
Activating project at 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project'
[ Info: Saved animation to C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\plots\philosophers_simulation.gif
Анимация сохранена в plots/philosophers_simulation.gif
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts>
```

Рисунок 8: Код анимации

Анимация процесса

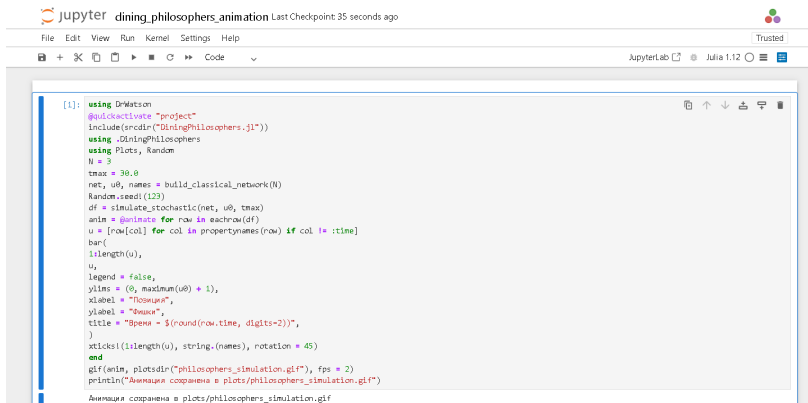
Запускаем и создаем все производные форматы(рис. 9).

```
Готово! Все файлы созданы.  
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts> julia tangle.jl dining_philosophers_animation.jl  
Activating project at 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project'  
Генерация из: dining_philosophers_animation.jl  
[ Info: generating plain script file from 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts\dining_philosophers_animation.jl'  
[ Info: writing result to 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts\dining_philosophers_animation\dining_philosophers_animation.jl'  
√ Чистый скрипт: C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts\dining_philosophers_animation\dining_philosophers_animation.jl  
[ Info: generating markdown page from 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts\dining_philosophers_animation.jl'  
[ Info: writing result to 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\markdown\dining_philosophers_animation\dining_philosophers_animation.qmd'  
√ Quarto: C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\markdown\dining_philosophers_animation\dining_philosophers_animation.qmd  
[ Info: generating notebook from 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts\dining_philosophers_animation.jl'  
[ Info: writing result to 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\notebooks\dining_philosophers_animation\dining_philosophers_animation.ipynb'  
√ Notebook: C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\notebooks\dining_philosophers_animation\dining_philosophers_animation.ipynb  
  
Готово! Все файлы созданы.  
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts>
```

Рисунок 9: производные форматы

Анимация процесса

Запускаем файл notebook(рис. 10).



The screenshot shows a JupyterLab window titled "dining_philosophers_animation" with a "Trusted" status. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for file operations and execution. The main area contains a code cell with the following Julia code:

```
[1]: using DelimitedFiles
@quickactivate "project"
include(scrdir("DiningPhilosophers.jl"))
using .DiningPhilosophers
using Plots, Random
N = 3
tmax = 30.0
net, u0, names = build_classical_network(N)
Random.seed!(123)
df = simulate_stochastic(net, u0, tmax)
anim = @animate for row in eachrow(df)
u = [row[col] for col in propertynames(row) if col != :time]
bar(
    1:length(u),
    u,
    legend = false,
    ylims = (0, maximum(u0) + 1),
    xlabel = "Позиция",
    ylabel = "Факты",
    title = "Время - $(round(row.time, digits=2))",
)
xticks(1:length(u), string.(names), rotation = 45)
end
gif(anim, plotsdir("philosophers_simulation.gif"), fps = 2)
println("Анимация сохранена в plots/philosophers_simulation.gif")

Анимация сохранена в plots/philosophers_simulation.gif
```

Рисунок 10: Код модели

Создаем и запускаем скрипт для итогового отчета и сравнительного анализа двух моделей(рис. 11).

```
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts>  
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts> julia dining_philosophers_report.jl  
Activating project at 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project'  
Отчёт сохранён в plots/final_report.png  
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts>
```

Рисунок 11: Запуск

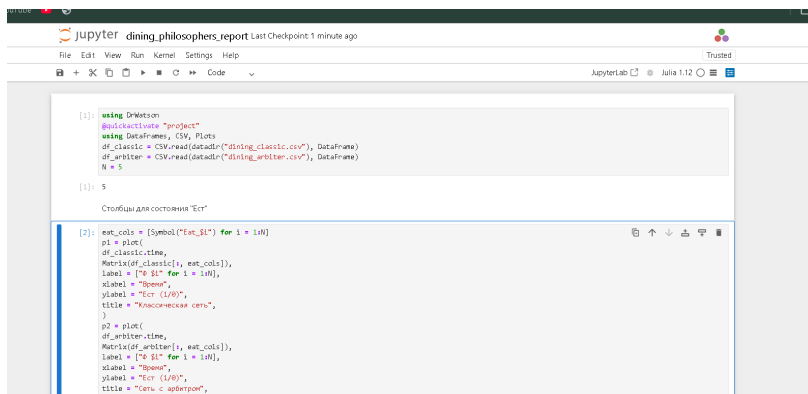
Создаем все производные форматы(рис. 10).

```
Отчет сохранен в plots/final_report.png
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts> julia tangle.jl dining_philosophers_report.jl
Activating project at 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project'
Генерация из: dining_philosophers_report.jl
[ Info: generating plain script file from 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts\dining_philosophers_report.jl'
[ Info: writing result to 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts\dining_philosophers_report\dining_philosophers_report.jl'
✓ Чистый скрипт: C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts\dining_philosophers_report\dining_philosophers_report.jl
[ Info: generating markdown page from 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts\dining_philosophers_report.jl'
[ Info: writing result to 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\markdown\dining_philosophers_report\dining_philosophers_report.qmd'
✓ Quarto: C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\markdown\dining_philosophers_report\dining_philosophers_report.qmd
[ Info: generating notebook from 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts\dining_philosophers_report.jl'
[ Info: writing result to 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\notebooks\dining_philosophers_report\dining_philosophers_report.ipynb'
✓ Notebook: C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\notebooks\dining_philosophers_report\dining_philosophers_report.ipynb

Готово! Все файлы созданы.
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts> |
```

Рисунок 12: производные форматы

Запускаем файл notebook(рис. 13).



```
[1]: using DrWatson
@quickactivate "project"
using DataFrames, CSV, Plots
df_classic = CSV.read(datadir("dining_classic.csv"), DataFrame)
df_arbiter = CSV.read(datadir("dining_arbiter.csv"), DataFrame)
N = 5

[1]: 5

Столбцы для состояния "Ест"

[2]: eat_cols = [Symbol("Eat_$(i)" for i = 1:N]
p1 = plot(
    df_classic.time,
    Matrix(df_classic[:, eat_cols]),
    label = ["0 $(i)" for i = 1:N],
    xlabel = "Время",
    ylabel = "Ест (1/0)",
    title = "Классическая сеть",
)
p2 = plot(
    df_arbiter.time,
    Matrix(df_arbiter[:, eat_cols]),
    label = ["0 $(i)" for i = 1:N],
    xlabel = "Время",
    ylabel = "Ест (1/0)",
    title = "Сеть с арбитром",
)
```

Рисунок 13: notebook

В результате получаем график (рис. 14).

Раздел 5

Выводы

После выполнения данной лабораторной работы мы познакомились с математическим аппаратом Сеть Петри и задачей об обедающих философах

Раздел 6

Список литературы

Список литературы